

UNIVERSIDAD COMPLUTENSE DE MADRID
FACULTAD DE CIENCIAS MATEMÁTICAS

DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y
COMPUTACIÓN



TRABAJO DE FIN DE GRADO

**Redes Neuronales y Algoritmos
Genéticos para Juegos de Interacción
en Tiempo Real**

Presentado por: Lorenzo García Bravo

Dirigido por: Rafael del Vado Vírseda

Grado en Matemáticas

Curso académico 2024-25

Agradecimientos

Este trabajo es fruto de mi curiosidad por el conocimiento, alimentada desde la infancia por mis padres dado que siempre que he tenido una pregunta o duda ha sido bien recibida y resuelta ampliamente por ellos. También quería agradecerle su apoyo a mis amigos, con quienes he compartido muy buenos momentos estos últimos años y, en particular, me gustaría agradecer su ayuda a Daniel Aragón, por permitirme compartir con él mis reflexiones sobre este trabajo. Finalmente, quiero darle las gracias a mi profesor y tutor de este trabajo, Rafael del Vado, quien siempre ha recibido con agrado mi propuesta de investigación y aportado fuentes que me han guiado en la realización del trabajo.

Resumen

Hoy en día la inteligencia artificial permite a las máquinas realizar tareas complejas que creíamos de carácter exclusivamente humano, desde el reconocimiento de imágenes y patrones hasta responder con lenguaje natural a las solicitudes de un usuario. Las redes neuronales son los modelos de predicción que operan por detrás, detectando patrones y generando respuestas pero... ¿cómo aprenden a realizar tales tareas? Las redes neuronales buscan emular el comportamiento del sistema nervioso central y, para especializarse en resolver un determinado problema, existen muchas técnicas de optimización de parámetros, entre ellas, los algoritmos genéticos, los cuales se basan en los fundamentos de la teoría de la evolución de las especies. El objetivo de este trabajo es investigar el funcionamiento de las redes neuronales, entender los algoritmos genéticos y hacerlas aprender a jugar a ciertos videojuegos para tener una comprensión visual tanto del funcionamiento de los algoritmos genéticos como de la evolución de las especies.

Palabras clave: *Algoritmo genético, aprendizaje supervisado, colisión, coste computacional, cruce, élite, etapa, evolución, estructura de datos, función de activación, función de aptitud, función de pérdida, generación, hiperplano, juego de interacción en tiempo real, modelo de optimización, modelo de predicción, mutación, perceptrón, pesos, problema de clasificación binaria, ruido gaussiano, sobreajuste, tasa de aprendizaje, sesgos.*

Abstract

Nowadays, artificial intelligence enables machines to perform complex tasks that we used to believe as exclusively human, from image and pattern recognition to responding in natural language to a user's requests. Neural networks are the prediction models that operate behind the scenes, detecting patterns and generating responses, but... how do they learn to perform such tasks? Neural networks aim to emulate the behavior of the central nervous system, and to specialize in solving a particular problem, there are many parameter optimization techniques, among them, genetic algorithms, which are based on the principles of the theory of evolution. The aim of this project is to investigate how neural networks work, understand genetic algorithms, and make them learn to play certain video games, in order to gain a visual understanding of both the functioning of genetic algorithms and the evolution of species.

Keywords: *Genetic algorithm, supervised learning, collision, computational cost, crossover, elite, stage, evolution, data structure, activation function, fitness function, loss function, generation, hyperplane, real-time interaction game, optimization model, prediction model, mutation, perceptron, weights, binary classification problem, gaussian noise, overfitting, learning rate, biases.*

Índice general

1. Introducción	6
1.1. Teoría de la evolución y algoritmos genéticos	6
1.2. Motivación y objetivos	7
1.3. Contexto y Antecedentes del trabajo	8
1.3.1. Tipos de aprendizaje	8
1.3.2. Juegos de interacción en tiempo real	8
1.3.3. <i>Software</i> y <i>Hardware</i>	9
1.4. Estructura de la memoria	10
2. Introducción a las redes neuronales	11
2.1. El Perceptrón	11
2.1.1. Funcionamiento del Perceptrón	11
2.1.2. Entrenamiento del perceptrón	12
2.1.3. Implementación en Python	14
2.2. Redes neuronales profundas: El perceptrón multicapa	15
2.2.1. Estructura y funcionamiento del MLP	15
2.2.2. Entrenamiento del MLP	16
3. Implementación del perceptrón multicapa	18
3.1. Primer enfoque: entrenamiento supervisado	18
3.2. Segundo enfoque: algoritmos genéticos	24
3.2.1. Cruce y mutación	26

3.2.2. Implementación	30
4. Conclusiones y Trabajo Futuro	36
4.1. Conclusiones	36
4.2. Trabajo futuro	37

Capítulo 1

Introducción

1.1. Teoría de la evolución y algoritmos genéticos

Cuando uno observa la dinámica de un ser vivo, se da cuenta de que existe cierta armonía entre sus características y los elementos del entorno con el que interactúa. Esta armonía, en ocasiones, se da gracias a unos parámetros favorables del hábitat (temperatura, humedad, recursos...) aunque, en otras ocasiones, es el propio ser vivo quien, gracias a un sofisticado diseño, es capaz de subsistir en entornos hostiles y hacer frente a las amenazas que le acechan.

Los seres vivos, desde los más simples hasta los más complejos, cuentan con complicados mecanismos que les permiten adaptarse a su entorno y así desempeñar las tres funciones vitales. Algunas de estas adaptaciones se manifiestan en comportamientos; un ejemplo llamativo es el expuesto en la serie de ‘Cosmos’ (2014) presentada por el astrofísico *Neil deGrasse Tyson*, donde explican que las abejas, al morir, segregan una sustancia llamada ácido oleico. Otras abejas, al detectar esta sustancia, retiran el cadáver de la abeja, manteniendo así la higiene dentro de la colmena y evitando la propagación de enfermedades. Esta reacción de las abejas es instintiva: incluso aquellas que nunca antes han realizado ni presenciado esta práctica, también la ejecutan al detectar el ácido oleico.

Entonces, ¿realmente tienen voluntad propia sobre las actividades que realizan? Si no es así, ¿cómo están programadas para realizar tales tareas sin ningún tipo de educación en su tiempo de vida? ¿Qué tareas realiza un ser vivo por instinto y cuáles por voluntad propia? El debate sobre dónde comienza la voluntad propia (sobre todo si lo extendemos a los seres humanos) es de carácter filosófico. No obstante, la **teoría de la evolución de Darwin** sí puede justificar que tales comportamientos instintivos sean innatos [1].

La teoría de la evolución de las especies de Charles Darwin propone que las adaptaciones que realizan los seres vivos a sus entornos son fruto de cambios aleatorios (mutaciones) en el material genético que define todo su ser.

Cuando se produce una mutación aleatoria que favorece a un individuo de alguna manera, esta le proporciona una mayor facilidad para sobrevivir y reproducirse que al resto de ejemplares de su especie, por lo que lentamente este gen favorable producirá un aumento

en la población que porta esta nueva información genética. Por el contrario, un cambio desfavorable o neutro, que suele ser el caso de la mayoría de las mutaciones genéticas, puede perjudicar al organismo y encaminarlo a la extinción. En definitiva, todos los seres vivos que encontramos a día de hoy son el resultado de la composición de mutaciones genéticas que han permitido la reproducción desde el inicio de la vida en la Tierra.

Los algoritmos genéticos son modelos de optimización que se basan en esta teoría evolutiva para encontrar de manera heurística la solución de un problema. Tanto la estructura de los algoritmos como el proceso de selección y la creación de nuevos individuos buscan imitar el *modus operandi* de la naturaleza, que siempre encuentra una forma de abrirse camino ante las adversidades.

1.2. Motivación y objetivos

Mi interés por los algoritmos genéticos surgió tras ver en Internet que había gente que usaba estos modelos de optimización para entrenar y ajustar los parámetros de ciertos modelos de predicción (como redes neuronales), con la finalidad de que aprendieran a jugar de manera óptima a algunos videojuegos. Fue en el año 2022 cuando intenté, por mi propia cuenta, implementar un algoritmo genético y entrenar a un modelo que jugara al famoso videojuego *Geometry Dash*, en el que están inspirados los juegos a los que se hará frente en este trabajo.

Ahora bien, para tener un acceso absoluto a los parámetros y variables del videojuego con los que poder implementar con facilidad el algoritmo genético, necesitaba disponer de su código fuente, al que nadie suele tener acceso. Una manera de eludir este problema y realizar, aun así, una implementación propia, es intentar replicar programando desde cero las partes del videojuego que me interesaban. En su día fue un reto personal pero, una vez programado, quedaba la parte más interesante: entrenar a un modelo de predicción que jugase, en concreto, una red neuronal.

Para sorpresa mía, todos los ejemplos que me habían inspirado hacían uso de la librería **Neat** [2] de Python, la cual trae implementado el algoritmo genético de tal forma que el usuario solo debe introducir los parámetros que desee para su problema. Así lo hice en un primer intento, y funcionó, pero no quedé satisfecho dado que la librería hacía prácticamente todo lo que me interesaba entender. Así que me propuse investigar tanto el funcionamiento de las redes neuronales como de los algoritmos genéticos y, finalmente, hacer una implementación propia.

Por tanto, el objetivo de este trabajo es hacer un recorrido desde los conceptos más básicos de redes neuronales convencionales, entender el funcionamiento de los algoritmos genéticos y entrenar ciertos modelos para que sean capaces de jugar a los juegos mencionados anteriormente.

1.3. Contexto y Antecedentes del trabajo

1.3.1. Tipos de aprendizaje

A día de hoy existen muchas maneras de entrenar a un modelo para resolver un problema o predecir su solución (modelos estadísticos, redes neuronales, *clustering*, árboles de decisión...) y se pueden clasificar según el nivel de intervención externa en el proceso de aprendizaje o ajuste de estos modelos:

- **Aprendizaje supervisado:** El modelo se ajusta a través de una serie de datos de los que se conoce la respuesta esperada del modelo (etiquetas). Se dice supervisado porque requiere de un conocimiento preciso de la respuesta esperada del modelo.
- **Aprendizaje no supervisado:** El modelo aprende de forma automática a base de buscar patrones en los datos, en este caso, sin etiquetar. Se le llama no supervisado puesto que explora el espacio de soluciones según la estructura propia de los datos, sin información adicional.
- **Aprendizaje por refuerzo:** El modelo se entrena a partir de unos datos sin etiquetar, pero un sistema de recompensas le ayuda a determinar la dirección en la que la solución mejora.

1.3.2. Juegos de interacción en tiempo real

Nos referiremos por juego de interacción en tiempo real [3] como aquel que no se detiene a la espera de una respuesta del jugador, sino que continúa actualizándose con una frecuencia elevada (por lo general, **fotogramas**). El tiempo de reacción y los reflejos toman mayor relevancia, lo que los distingue de juegos de turnos (ajedrez, tres en raya, *mastermind*).

Dada la libertad del jugador para reaccionar o realizar una acción en cada fotograma, estos juegos se caracterizan por la inmensa cantidad de estados o situaciones diferentes que se generan. Como consecuencia, encontrar una estrategia ganadora se convierte en una tarea compleja debido a la gran cantidad de parámetros que deben tenerse en cuenta. Por esta razón, en muchos casos se recurre a las **redes neuronales**, que en estos juegos suelen optimizarse de dos maneras: mediante un conjunto de datos de entrenamiento (utilizando técnicas como el **descenso del gradiente** o el optimizador Adam) o bien sin datos de entrenamiento, a través de **algoritmos genéticos**. En este último caso, la optimización resulta más flexible, ya que permite explorar más ampliamente todas las posibles soluciones.

En este trabajo examinaremos dos juegos de elaboración propia, programados en **Python** a través de una librería de dibujado gráfico **Pygame** [4], la cual proporciona un lienzo sobre el que se pueden dibujar figuras básicas o píxeles concretos. No obstante, la actualización de fotogramas mediante el bucle principal del juego, reglas y físicas, queda en manos del usuario que quiera hacer uso de la librería.

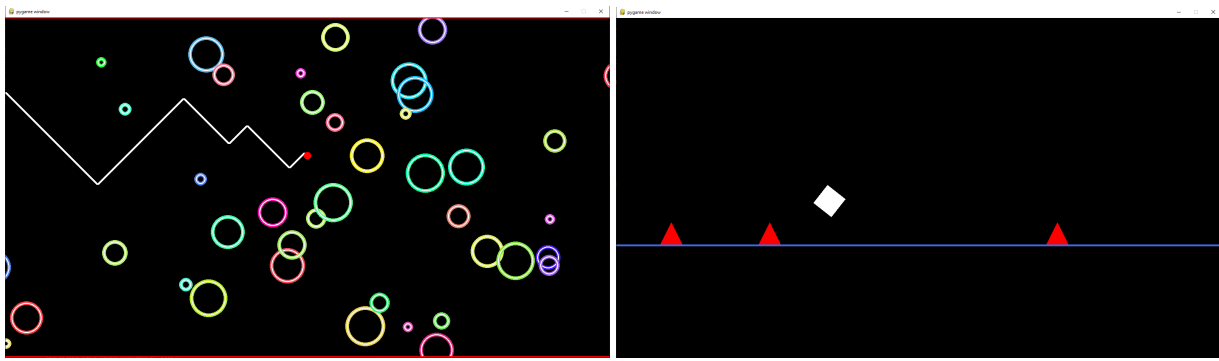


Figura 1.1: Juegos de elaboración propia programados en pygame inspirados en dos modos de juego de *Geometry Dash*. La imagen de la izquierda busca imitar lo que se conoce como ‘wave’ y el de la izquierda el modo ‘cubo’.

En el desarrollo de los dos juegos, aunque no sea competencia de este trabajo, cabe mencionar el inherente uso de matemáticas y técnicas de optimización donde el uso de diseño de algoritmos, programación dinámica y programación paralela (para la gestión de obstáculos y generaciones) ha mejorado, versión tras versión, la fluidez de los juegos, permitiendo una tasa de refresco de 120 fotogramas por segundo, y un entrenamiento de 500 modelos de forma simultánea (en el algoritmo genético) en tiempos de ejecución razonables a nivel doméstico.

1.3.3. *Software y Hardware*

Para la realización de este trabajo, la implementación de los algoritmos se ha realizado en **Python** (v.3.8.0 - v.3.13.0). A su vez, se ha hecho uso de algunas librerías básicas como *math*, *time* y *random*. El resto de las librerías se mencionarán y explicarán conforme se vayan usando a lo largo del trabajo.

Cabe destacar que todas las imágenes incorporadas en la memoria son de elaboración propia. Aquellas imágenes que no son capturas de los juegos, han sido realizadas a través del editor de gráficos vectoriales de *software* libre **Inkscape**.

Por otro lado, el *hardware* utilizado es un portátil *MSI GT73VR 6RE Titan* de 2017, con un CPU Intel Core i7-6700HQ (8 CPUs) de 2,6 GHz ampliables a 3,5 GHz y una tarjeta gráfica NVIDIA GeForce GTX 1070 utilizada a través de CUDA junto al procesador para procesar el algoritmo genético.

1.4. Estructura de la memoria

La memoria se estructura del modo siguiente:

- En el presente **Capítulo 1** hemos presentado el tema de este trabajo y expuesto la teoría de la evolución para una mejor comprensión de la motivación de los algoritmos genéticos, así como una clasificación de los tipos de entrenamientos de modelos.
- El **Capítulo 2** se centra en realizar un recorrido sobre las redes neuronales desde el perceptrón (comentando sus limitaciones y realizando una implementación) a la red neuronal convencional.
- En el **Capítulo 3** realizamos dos implementaciones de la red neuronal convencional con dos enfoques de entrenamiento diferentes. Un entrenamiento supervisado y un entrenamiento a través de un algoritmo genético en dos juegos inspirados en el videojuego *Geometry Dash*.
- Por último, en el **Capítulo 4** se presentan las conclusiones finales, el trabajo futuro y algunos trabajos publicados relacionados con la investigación realizada.

A lo largo de esta memoria se presentan diferentes cuadros de código que sirven para apoyar las experimentaciones realizadas y las explicaciones textuales. Sin embargo, la totalidad del código implementado puede encontrarse en el siguiente repositorio de GitHub:

<https://github.com/Difeomorfismo/TFG-Lorenzo.git>.

Capítulo 2

Introducción a las redes neuronales

En este capítulo se estudiará la estructura y funcionamiento del Perceptrón, la unidad básica de las redes neuronales y también el perceptrón multicapa, el formato clásico de red neuronal profunda, capaz de afrontar bastantes más problemas que el perceptrón a base de una estructura y entrenamiento más complejos.

El contenido del presente capítulo ha sido extraído de los siguientes artículos: [5], [6], [7] y [8]. Aunque algunos conceptos que se exponen a continuación han sido reelaborados de forma propia a partir de dichas fuentes.

2.1. El Perceptrón

En el año 1958, Frank Rosenblatt (1928-1971), tras estar al frente de varias investigaciones en neurobiología, desarrolló el perceptrón imitando el comportamiento de una neurona humana. En una red neuronal, el perceptrón es el equivalente a una neurona en el sistema nervioso y, de hecho, estructuralmente tiene un funcionamiento similar. Ambas reciben una o varias variables de entrada y, tras un proceso, emiten una salida.

A continuación, veremos en mayor profundidad el funcionamiento de un perceptrón, qué problemas puede afrontar y cuáles no, la manera en la que puede aprender y, finalmente, haremos una implementación en Python.

2.1.1. Funcionamiento del Perceptrón

Un perceptrón [6] recibe una entrada, supongamos que de n variables, $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Para procesar la entrada, el perceptrón tiene asociado a cada variable x_i un peso w_i del vector $\mathbf{w} = (w_1, w_2, \dots, w_n)$. Para calcular la salida se suman los productos de cada variable por su peso, y al resultado se le suma un sesgo b , es decir, se calcula el valor $z = \mathbf{x} \cdot \mathbf{w} + b$.

A continuación, queda determinar si el perceptrón se activa o no según el valor de z . Para esto introducimos la **función de activación** que, por lo general, para el caso del perceptrón simple, se toma la función:

$$f(z) = \begin{cases} 1, & \text{si } z \geq u \\ 0, & \text{si } z < u \end{cases} \quad \text{para cierto umbral } u. \quad (2.1)$$

Como podemos observar, el perceptrón es un algoritmo de clasificación binaria, es decir, determina si un vector pertenece o no a cierta clase. En concreto, separa el espacio vectorial de las entradas a través del hiperplano $H \equiv \mathbf{x} \cdot \mathbf{w} + b - u = 0$. Para que el perceptrón pueda clasificar de forma correcta, es necesario que las clases sean linealmente separables, es decir, que exista un hiperplano que las separe. En caso contrario, el perceptrón no será capaz de clasificar de forma correcta, como veremos más adelante.

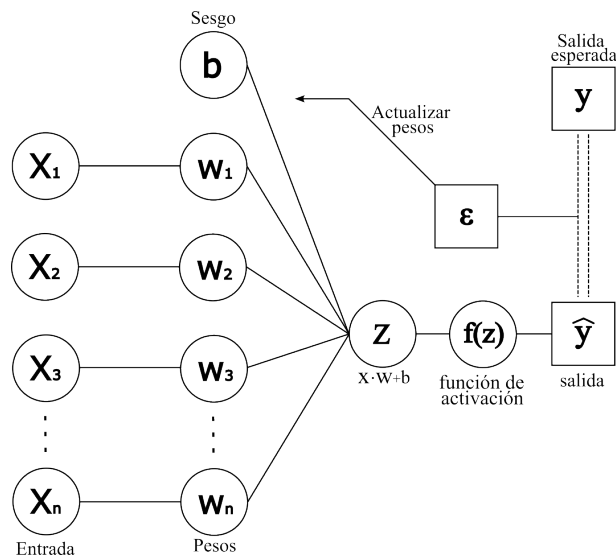


Figura 2.1: Esquema del funcionamiento de un Perceptrón. Fuente: Elaboración propia.

2.1.2. Entrenamiento del perceptrón

La utilidad de esta estructura reside en la capacidad que tiene de ajustarse a un problema determinado a base de reajustar el vector de pesos $\mathbf{w}$ y, aunque se podría determinar el valor de los pesos planteando un problema de programación lineal, existe una forma de que el perceptrón aprenda a partir de una fase de entrenamiento. Esta fase consiste en evaluar el perceptrón con diferentes vectores y corregir los errores que comete de forma supervisada con las respuestas que deseáramos obtener.

La forma de ajustar los pesos es la siguiente. Supongamos que al evaluar un vector $\mathbf{x}$ deseamos obtener una salida y , sin embargo, el perceptrón nos devuelve el valor $\hat{y}$. La forma de reajustar los pesos es:

$$w_i := w_i + x_i \cdot \eta \cdot \epsilon$$

$$b := b + \eta \cdot \epsilon$$

Donde $\varepsilon = y - \hat{y}$ es el error cometido y η es la **tasa de aprendizaje**, que representa el ritmo al que se ajustan los pesos. Una tasa de aprendizaje alta puede hacer que el algoritmo nunca converja, mientras que una tasa demasiado baja puede hacer que tarde mucho tiempo en converger.

La idea es ir iterando sobre los datos de entrenamiento hasta que la red deje de aprender (el error sea 0 para todos los casos). A cada iteración sobre los datos de entrenamiento se le llama **etapa** (o **época**).

Como hemos mencionado anteriormente, realmente lo que hace el perceptrón es encontrar un hiperplano que separe los datos en dos clases. Si las clases que deseamos separar no pueden ser separadas con un único hiperplano, el entrenamiento nunca convergerá.

Un ejemplo típico es el caso de las operaciones binarias *AND*, *OR* y *XOR*. Mientras que *AND* y *OR* sí se pueden entrenar en un perceptrón, *XOR* no. Una manera visual de entender esto es con los siguientes gráficos:

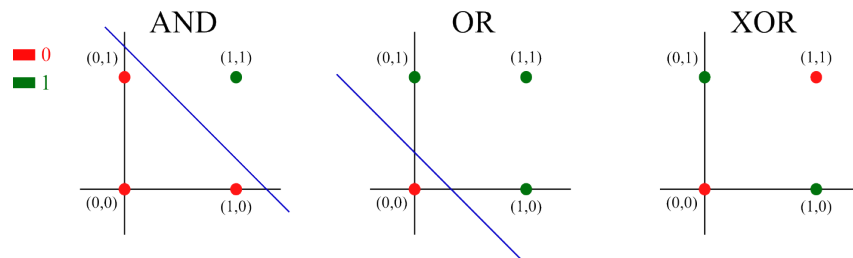


Figura 2.2: Resolución visual del problema de clasificación binaria de *AND*, *OR* y *XOR*. Fuente: Elaboración propia.

Observación 2.1.2.1. *Un perceptrón simple no puede ajustar sus pesos para resolver la función XOR.*

Demostración. Efectivamente, si el perceptrón hallara unos pesos w_i y un sesgo b con las funciones lineales que usa, se tendría, con la función de activación [2.1](#) tomando $u = 0$ (tomaremos este valor de u de ahora en adelante dado que no influye en la existencia del hiperplano):

$$\begin{cases} f(0 \cdot w_1 + 0 \cdot w_2 + b) = 0 \\ f(1 \cdot w_1 + 0 \cdot w_2 + b) = 1 \\ f(0 \cdot w_1 + 1 \cdot w_2 + b) = 1 \\ f(1 \cdot w_1 + 1 \cdot w_2 + b) = 0 \end{cases} \implies \begin{cases} b < 0 \\ w_1 + b \geq 0 \\ w_2 + b \geq 0 \\ w_1 + w_2 + b < 0 \end{cases} \implies \begin{cases} b < 0 \\ w_1 + w_2 \geq -2b \\ -2b + b < 0 \end{cases}$$

Pero entonces tenemos $b < 0$ y que $-b < 0$, que es una contradicción. $\square$

Por lo tanto, efectivamente, el perceptrón no puede resolver *XOR* ni ninguno que requiera de más de un hiperplano para realizar la clasificación. Esto limita bastante al perceptrón y es por ello por lo que se buscarán maneras de añadir complejidad a la estructura para sortear el problema.

2.1.3. Implementación en Python

Para implementar el perceptrón, lo que haremos será definir una clase para gestionar la estructura del mismo. Guardaremos tanto los pesos, el sesgo, la tasa de aprendizaje, etc., así como las funciones de activación, evaluación del perceptrón y entrenamiento del perceptrón con un conjunto de datos de entrenamiento.

En el siguiente código se hace uso de la librería **numpy** para un rápido cálculo de los productos escalares, sobre todo, en la línea 12:

```

1 import numpy as np
2
3 class Perceptron:
4     def __init__(self, dimension, tasa_aprendizaje=0.01, etapas=5):
5         self.pesos = np.zeros(dimension)
6         self.sesgo = 0
7         self.tasa_aprendizaje = tasa_aprendizaje
8         self.etapas = etapas
9     def funcion_de_activacion(self, x):
10        return 1 if x >= 0 else 0
11    def evaluar(self, x):
12        z = self.pesos.T.dot(x) + self.sesgo
13        return self.funcion_de_activacion(z)
14    def entrenar(self, X, Y):
15        for _ in range(self.etapas):
16            for x, y in zip(X, Y):
17                y_gorro = self.evaluar(x)
18                error=(y - y_gorro)
19                self.pesos += self.tasa_aprendizaje * error * x
20                self.sesgo+=self.tasa_aprendizaje * error

```

A continuación, para entrenar y poner a prueba el perceptrón, por ejemplo, con la operación *AND*:

```

1 X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
2 Y = np.array([0, 0, 0, 1])
3
4 perceptron = Perceptron(dimension=2)
5 perceptron.entrenar(X, Y)
6
7 # Pruebas
8 print('[0, 0] -> '+str(perceptron.evaluar([0, 0])))
9 print('[0, 1] -> '+str(perceptron.evaluar([0, 1])))
10 print('[1, 0] -> '+str(perceptron.evaluar([1, 0])))
11 print('[1, 1] -> '+str(perceptron.evaluar([1, 1])))

```

Los resultados que se obtienen son satisfactorios:

```

1 [0, 0] -> 0
2 [0, 1] -> 0
3 [1, 0] -> 0
4 [1, 1] -> 1

```

No obstante, como hemos mencionado anteriormente, para XOR el algoritmo nunca termina de converger, por lo que, una vez recorridas todas las etapas, el algoritmo dará una solución errónea.

Esto supone un problema a la hora de plantear casos de mayor complejidad. Para algunos problemas, sería suficiente hacer la composición con otros perceptrones (añadir capas). Esto no siempre es suficiente cuando se trata de problemas no lineales, por esto, la función de activación también debe ser modificada. Es por ello por lo que se introducen las redes neuronales profundas, en particular, el **perceptrón multicapa**.

2.2. Redes neuronales profundas: El perceptrón multicapa

Dado que el perceptrón simple era incapaz de afrontar problemas no linealmente separables, se intentó, desde el principio, conectar varios perceptrones en diversas capas. El problema es que se carecía de tecnología para realizar de manera eficiente el ajuste de pesos. Hubo que esperar hasta el año 1980 cuando se creó el algoritmo de retropropagación que permitía optimizar los pesos.

2.2.1. Estructura y funcionamiento del MLP

La estructura de un perceptrón multicapa está dividida en capas de perceptrones (o neuronas). Una MLP tiene una primera capa que recogerá las variables de entrada. A continuación, las siguientes capas son de tamaño variable, a las que llamaremos capas ocultas. Están formadas por neuronas cuyas entradas son los valores de salida de las neuronas de la capa anterior y, además, cada una de ellas tiene sus propios pesos y una función de activación de la que hablaremos más adelante. Finalmente, la información se computa a través de las diferentes neuronas avanzando por cada capa hasta llegar a la última de ellas, la capa de salida, donde encontraremos el resultado que da la red neuronal a nuestra entrada inicial.

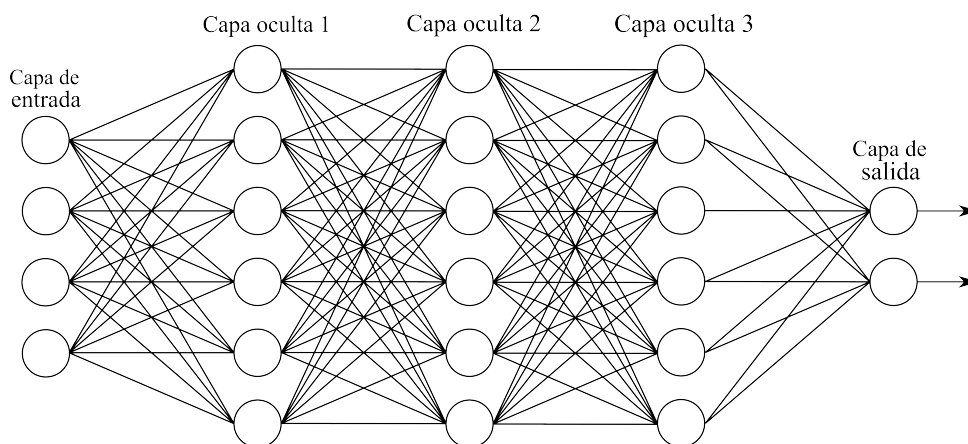


Figura 2.3: Esquema en forma de grafo de un perceptrón multicapa formado por una capa de entrada de 4 neuronas, tres capas ocultas de 6 neuronas cada una y, finalmente, una capa de salida de 2 neuronas. Fuente: Elaboración propia.

Llamaremos $\mathbf{l}_i$ a los valores calculados en la capa $i \in \{0, \dots, L\}$, donde $\mathbf{l}_0$ es la entrada $\mathbf{x}$ y $\mathbf{l}_L$ es la salida $\hat{\mathbf{y}}$. Como cada capa tiene un tamaño variable de neuronas, nos referiremos por n_i al número de neuronas que tiene la capa $\mathbf{l}_i$.

A continuación, en una capa $\mathbf{l}_i$, cada neurona calcula el producto escalar de todos los valores de la capa $\mathbf{l}_{i-1}$ por el vector de pesos de la neurona, sumará el sesgo y evaluará dicho valor en su función de activación.

Ahora bien, esta operación capa por capa puede verse de forma matricial, agrupando por filas los vectores de pesos de todas las neuronas de la capa, construyendo la matriz $\mathbf{W}_i$ de tamaño $n_i \times n_{i-1}$ y el vector de los sesgos $\mathbf{b}_i$ de tamaño n_i . De esta forma, se tiene:

$$\mathbf{l}_i = f_i(\mathbf{W}_i \cdot \mathbf{l}_{i-1} + \mathbf{b}_i) \quad (2.2)$$

Aquí, $f_i : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación en la capa i , que se aplica coordenada a coordenada $f_i(\mathbf{x}) = (f_i(x_1), \dots, f_i(x_{n_i}))$. Es importante que esta función no sea lineal, pues de lo contrario, tendríamos los mismos problemas que tenía el perceptrón simple. La elección de la función de activación en cada capa depende mucho del problema que se quiera afrontar. Por lo general, se buscan funciones diferenciables para facilitar la optimización de los pesos.

Para las capas ocultas es muy común utilizar la función $ReLU(z) = \max(0, z)$ [9], aunque existen otras variantes que son útiles dependiendo del contexto del problema que se quiera afrontar (*LeakyReLU* o *ELU*).

Para la capa de salida, es importante saber escoger la función de activación correcta. Para una clasificación binaria, se utiliza la función **sigmoide** ($\sigma(z) = \frac{1}{1+e^{-z}}$) o la **tangente hiperbólica**. Para problemas de clasificación multiclase, es mejor usar la función **softmax**.

2.2.2. Entrenamiento del MLP

Existen muchas maneras de optimizar los parámetros de una red neuronal. Partiendo de una serie de datos etiquetados donde la respuesta a cada dato es conocida, un primer enfoque es similar al del perceptrón. Se evalúa la red neuronal, se calcula el error de la estimación y se ajustan los pesos en una dirección de mejora, minimizando el error.

La primera diferencia clave con el perceptrón simple es que este actualizaba sus pesos con cada dato que se leía, esto es lo que se conoce como **aprendizaje en línea** (online) [10]. Sin embargo, el enfoque que se le da al MLP es el **aprendizaje en lotes** (offline), es decir, que se actualizan los pesos tras evaluar todo el lote de datos de entrenamiento. Precisamente es ahora cuando cobra más sentido dividir el entrenamiento por etapas como hacíamos en el perceptrón simple. Ahora, cada etapa corresponde a una evaluación de los datos de entrenamiento.

En una etapa, una vez evaluados todos los datos de entrenamiento (digamos que hay N datos), las estimaciones $\hat{\mathbf{y}}_i$ que da la red neuronal a cada dato $\mathbf{x}_i$ se almacenan en el vector $\hat{\mathbf{y}}$. Este vector se compara con el vector de etiquetas $\mathbf{y}$, que contiene las soluciones $\mathbf{y}_i$ de cada dato. La manera de compararlo es con una función de error o **función de pérdida**.

La función de error más conocida es el **error cuadrático medio**:

$$MSE(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{i=0}^N (y_i - \hat{y}_i)^2 \quad (2.3)$$

Aunque existen otras funciones de pérdida útiles como la **pérdida de entropía cruzada** o **error logarítmico**, entre muchas otras como se puede ver en [11]. Nos referiremos a la función de pérdida como $L(y_i, \hat{y}_i)$.

Queda ahora ajustar los pesos en una dirección en la que se minimice la función de pérdida. El **método de descenso del gradiente** consiste en hallar el gradiente de $L(y_i, \hat{y}_i)$ en cada etapa, en función de la matriz de pesos W usada para hallar cada $\hat{y}_i$. Dicha dirección se multiplica por la **tasa de aprendizaje** η y se obtiene la dirección en la que se deben actualizar los pesos:

$$W' = W - \eta \nabla_W L(W) \quad (2.4)$$

Desafortunadamente, al sustituir cada valor $\hat{y}_i$ por la expresión [2.2], componer con todas las funciones de activación de cada capa nos queda:

$$\hat{y}_i = f_L(W_L f_{L-1}(W_{L-1} \dots f_1(W_1 x + b_1) + \dots + b_{L-1}) + b_L) \quad (2.5)$$

Reemplazar este valor en la función de pérdida $L(y_i, \hat{y}_i)$ y diferenciar en función de los pesos usando la regla de la cadena para calcular el gradiente de forma explícita es computacionalmente ineficiente. Por lo general, se opta por el método de **retropropagación** (backpropagation) u otros métodos más avanzados como el **optimizador Adam**.

El método de retropropagación [12] consiste en aplicar el descenso de gradiente capa por capa. Como la cantidad de combinaciones que se generan al aplicar la regla de la cadena en todas las capas a la vez, este método busca hacerlo capa por capa, utilizando los gradientes de la función de pérdida calculados en una capa para calcular los de la siguiente.

Como primer paso, se calcula el gradiente de la función de pérdida en la última capa, la capa de salida. Si $z_i = W_i \cdot l_{i-1} + b_i$:

$$\frac{\partial L}{\partial W_L} = \frac{\partial L}{\partial l_L} \cdot \frac{\partial l_L}{\partial z_L} \cdot \frac{\partial z_L}{\partial W_L} = \frac{\partial L}{\partial l_L} \cdot f'_L(z_L) \cdot l_{L-1}^T \quad (2.6)$$

Dado que:

$$\frac{\partial l_L}{\partial z_L} = f'_L(z_L) \quad \text{y} \quad \frac{\partial z_L}{\partial W_L} = \frac{\partial (W_L l_{L-1} + b_L)}{\partial W_L} = l_{L-1}^T$$

Entonces $\delta^{(L)} = \frac{\partial L}{\partial l_L} \cdot f'_L(z_L)$ mide la magnitud del error que producen los pesos de la última capa.

A partir de ahora, para una capa cualquiera, aplicando la regla de la cadena queda:

$$\frac{\partial L}{\partial W_i} = (W_{i+1}^T \cdot \delta_{i+1}) \cdot f'_i(z_i) \cdot l_{i-1}^T = \delta_i \cdot l_{i-1}^T \quad (2.7)$$

Una vez obtenidos todos los gradientes de cada capa, se actualizan los pesos con determinada tasa de aprendizaje η :

$$W_i = W_i - \eta \cdot \frac{\partial L}{\partial W_i}$$

Capítulo 3

Implementación del perceptrón multicapa

Para poner a prueba el MLP en **Python** utilizaremos las librerías de **pytorch** [13], dado que nos proporciona una implementación que nos da acceso a todos los parámetros de la red (capas, pesos, funciones de activación, optimizadores, funciones de cálculo de errores...). A continuación, se muestra un ejemplo de la estructura principal de la red:

```
1 class Red(nn.Module):
2     def __init__(self):
3         super(Red, self).__init__()
4         self.fc1 = nn.Linear(2, 10)
5         self.fc2 = nn.Linear(10, 10)
6         self.fc3 = nn.Linear(10, 1)
7
8     def forward(self, x):
9         x = torch.relu(self.fc1(x))
10        x = torch.relu(self.fc2(x))
11        x = torch.sigmoid(self.fc3(x))
12        return x
```

En este caso, definimos el tamaño de la red con las funciones **nn.Linear(a, b)**, que calculan en cada capa el valor $\mathbf{z}_i = \mathbf{W}_i \cdot \mathbf{l}_{i-1} + \mathbf{b}_i$, con $n_{i-1} = a$ y $n_i = b$. La función **forward** básicamente calcula la salida de la red dada una entrada $\mathbf{x}$, haciendo la composición de todas las funciones de activación de las capas. Toda la estructura de pesos y sesgos la hereda de la superclase **nn.Module** de Pytorch.

3.1. Primer enfoque: entrenamiento supervisado

Para el entrenamiento de la red, supongamos que tenemos un conjunto de datos de entrenamiento, de forma que en cada fila encontramos tanto los valores de entrada como los valores de salida esperados. Se trata de elegir una función de pérdida, un método optimizador (y con ello una tasa de aprendizaje $\mathbf{lr}$) y un número de etapas que son las veces que se actualizan los pesos.

```

1 model = Red()
2 funcion_perdida = nn.MSELoss() #error cuadrático medio
3 optimizador = optim.Adam(model.parameters(), lr=0.01)
4 entradas, y_esperado = get_data()
5 etapas=1000
6 for _ in range(etapas):
7     optimizador.zero_grad()
8     y_red = model(entradas)
9     error = funcion_perdida(y_red, y_esperado)
10    error.backward()
11    optimizador.step()

```

En nuestro caso, utilizaremos el error cuadrático medio [2.3](#) y el optimizador Adam, realizaremos 1000 iteraciones sobre los datos y usaremos una tasa de aprendizaje $\eta = 0,01$.

El problema que queremos que haga frente es aprender a jugar a un juego de elaboración propia, en el que el jugador debe esquivar ciertos obstáculos:

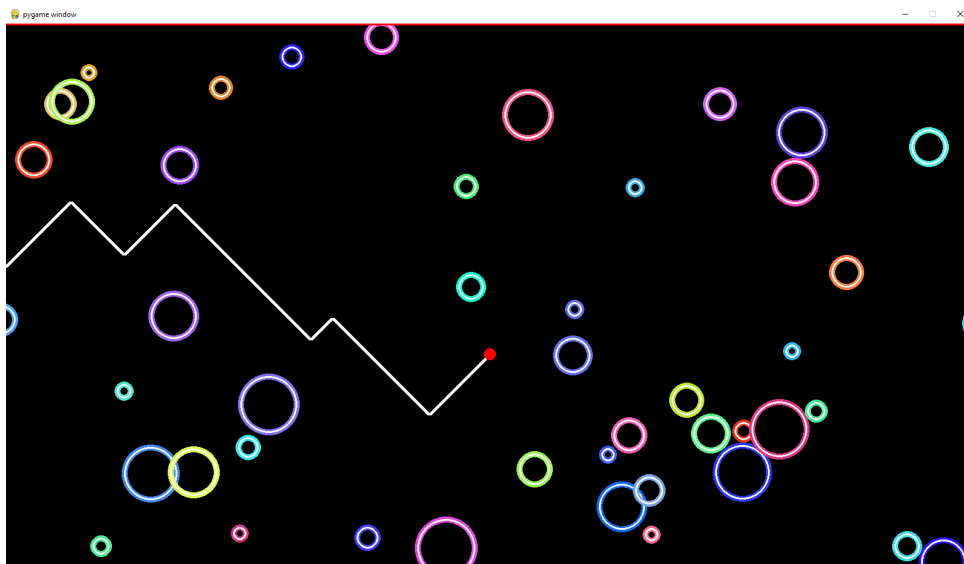


Figura 3.1: Captura del juego inspirado en el ‘wave’ del *Geometry Dash*, el jugador es el punto rojo que deja un rastro blanco en el camino realizado, los círculos de colores son los obstáculos que el jugador debe esquivar, generados de forma dinámica y aleatoria en el extremo derecho de la pantalla (distribución uniforme)

Todo el código del juego se puede encontrar en el repositorio de GitHub: <https://github.com/Difeomorfismo/TFG-Lorenzo.git>

El juego avanza hacia la derecha automáticamente, el jugador solo tiene dos opciones, o bien dar ‘click’ para que el jugador comience a subir verticalmente, o bien no hacer nada, por lo que el jugador bajará automáticamente. La velocidad a la que avanzan los obstáculos horizontalmente es la misma a la que se mueve el jugador verticalmente, de esta forma, el rastro del jugador forma ángulos rectos cada vez que cambia su dirección. Si el jugador toca un obstáculo, el suelo o el techo, el jugador perderá y desaparecerá.

Utilizando el código anterior, lo primero que necesitamos son datos de entrenamiento, pero antes, debemos analizar exactamente qué datos son los que debemos elegir.

En un primer momento, la idea era pasarle las coordenadas de todos los obstáculos para que tuviera control absoluto del juego, pero, desde luego, esto es inviable, primero porque la entrada es de tamaño variable según la cantidad de obstáculos y, segundo, porque la red tardaría bastante en generalizar.

Un segundo intento era pasarle una lista con las n distancias más pequeñas a los obstáculos que estuvieran por delante del jugador. Esto tampoco sirve, dado que hay situaciones en las que el jugador, tome la decisión que tome, va a esquivar un obstáculo muy cercano al que le está dando mucha atención por estar cerca, aunque no sea una amenaza, como se muestra a continuación:

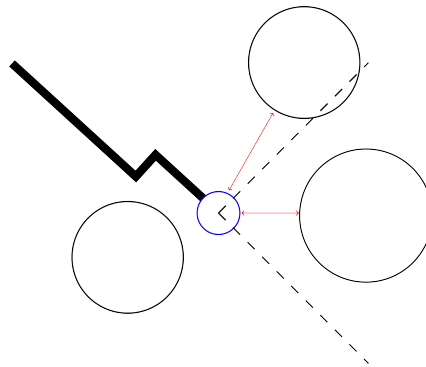


Figura 3.2: Ejemplo esquemático del juego donde el obstáculo más cercano no obstruye las trayectorias del jugador.

Por tanto, la red no distinguiría entre un obstáculo que supone un peligro y uno que no. Además, en esta imagen también se puede apreciar cómo la distancia al obstáculo de arriba no es la distancia real de choque, dado que la trayectoria que mide la distancia no es la que sigue el jugador. Esto nos da una pista clave para el siguiente y último enfoque: Ver a qué distancia cortan con un obstáculo las rectas que trazaría el jugador en ambos casos.

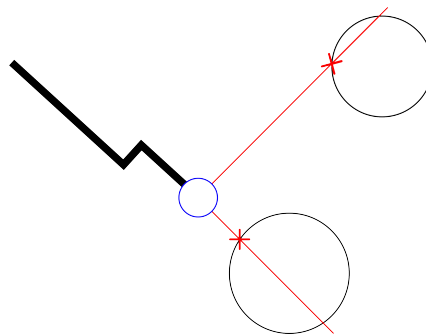


Figura 3.3: Esquema de las distancias reales a los obstáculos siguiendo la trayectoria del jugador.

Como en el bucle principal del juego hay que recorrer todos los obstáculos para actualizarlos cada fotograma, para calcular estas colisiones, tomaremos las rectas de pendientes 1 y -1 , hallaremos la intersección recta-circunferencia y, como puede haber dos

puntos de corte, tomaremos el más cercano. Considerando $CentroJugador = (a, b)$, $CentroObstaculo = (c, d)$, $RadioObstaculo = r$ y $Pendiente = m$ (valdrá 1 o -1) tenemos:

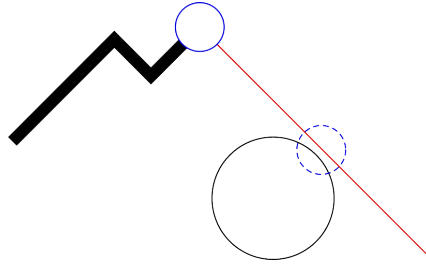
$$\begin{cases} recta \equiv y - a = m(x - b) \\ circunferencia \equiv (x - c)^2 + (y - d)^2 = r^2 \end{cases}$$

Tras hacer las cuentas nos queda:

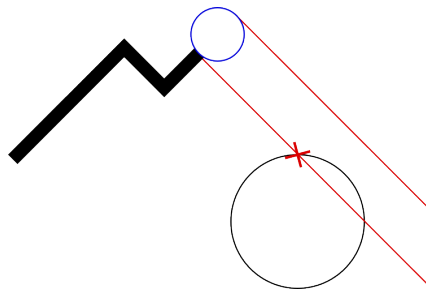
$$\begin{cases} x = \frac{2(c+k) \pm \sqrt{4((2ck)+2r^2-c^2-k^2)}}{4} \\ y = x - a + b \end{cases}, \text{ donde } k = a + m \cdot (b - d) \quad (3.1)$$

Que son las coordenadas de los puntos donde colisionaría el jugador en cada decisión. Como salen dos puntos de corte para cada circunferencia con la que puede colisionar, tomaremos el que tenga la menor distancia con el jugador. En caso de que no haya ningún obstáculo con la que chocaría, se pasará como dato la distancia al techo o suelo en cada caso.

Un problema que surge de esta forma es que, aunque la recta que estamos estudiando no colisione con cierto obstáculo, el jugador sí podría hacerlo, dado que este se modela como un círculo con un radio determinado.



Esto hace que, por los datos que recogemos, la red no tenga información acerca de obstáculos que sí son un peligro; por ello, en vez de tomar la recta que pasa por el centro del jugador, tomaremos las tangentes al jugador:



Tenemos que, los 4 puntos de tangencia de las rectas de pendiente $m \in \{-1, 1\}$ con el jugador de centro (a, b) y radio $\hat{r}$, son:

$$\begin{cases} x = a \pm \frac{\sqrt{2}}{2} \hat{r} \\ y = b \mp m \frac{\sqrt{2}}{2} \hat{r} \end{cases} \quad (3.2)$$

Sobre los que redefiniremos las rectas anteriores y calcularemos las posibles colisiones de las cuatro nuevas rectas. Finalmente, tenemos las funciones:

- *hay_corte* (línea 2). Evalúa el signo del discriminante de la ecuación de grado 2 (línea 14) para saber si las 4 rectas cortan las circunferencias, (las rectas son distinguidas por los parámetros 'm' e 'i' en el bucle de la línea 10, donde se define el punto por el que pasa cada recta).
- *distancia_choque* (línea 17). Calcula las distancias a los obstáculos de aquellas rectas que sí cortan a alguna circunferencia, haciendo uso de la expresión 3.1 (líneas 27-30). En caso de no cortar a ninguna circunferencia, se pone de forma temporal el valor 'infinito' ya que fuera de la función se tomará el mínimo entre las distancias calculadas y las distancias al borde de la pantalla (no se incluye esta comparación en esta función puesto que necesita información del ancho y alto de la pantalla).

```

1 from math import sqrt,inf
2 def hay_corte(a,b,pr,c,d,r,m):
3 # (a,b): centro y radio del jugador
4 # pr : radio del jugador
5 # (c,d): centro del obstaculo
6 # r : radio del obstaculo
7 # m : pendiente de la recta que se estudia (1 o -1)
8 # i : cada una de las dos rectas tangentes de pendiente m
9 corte=[0,0]
10 for i in [-1,1]:
11     x=a-i*sqrt(2)/2*pr
12     y=b-m*i*sqrt(2)/2*pr
13     k=x+m*(y-d)
14     corte[int(i/2+1/2)] = (2*c*k)+(2*r**2)-(c**2)-(k**2)>=0
15 return corte
16
17 def distancia_choque(a,b,pr,c,d,r,m,corte):
18 # corte es una lista que indica cuales de las tangentes corta
19 distancias=[]
20 for i in [-1,1]:
21     if corte[int(i/2+1/2)]==0:
22         distancias.append(inf)
23     else:
24         x=a-i*sqrt(2)/2*pr
25         y=b-m*i*sqrt(2)/2*pr
26         k=x+m*(y-d)
27         x1=(2*(c+k)+sqrt(4*((2*c*k)+(2*r**2)-(c**2)-(k**2))))/4
28         y1=x1-x+y
29         x2=(2*(c+k)-sqrt(4*((2*c*k)+(2*r**2)-(c**2)-(k**2))))/4
30         y2=x2-x+y
31         if x1>=x:
32             d1=(x-x1)**2+(y-y1)**2
33         else:
34             d1=inf
35         if x2>=x:
36             d2=(x-x2)**2+(y-y2)**2
37         else:
38             d2=inf
39         distancias.append(min(d1,d2))
40 return sqrt(min(distancias))

```

A continuación, necesitamos generar datos de entrenamiento. Para ello, en cada fotograma, se calcularán y guardarán en un documento de texto los datos mencionados anteriormente, etiquetados con la decisión que tome un jugador humano que sepa jugar. También se ha implementado una manera visual de ver los datos que se recogen:

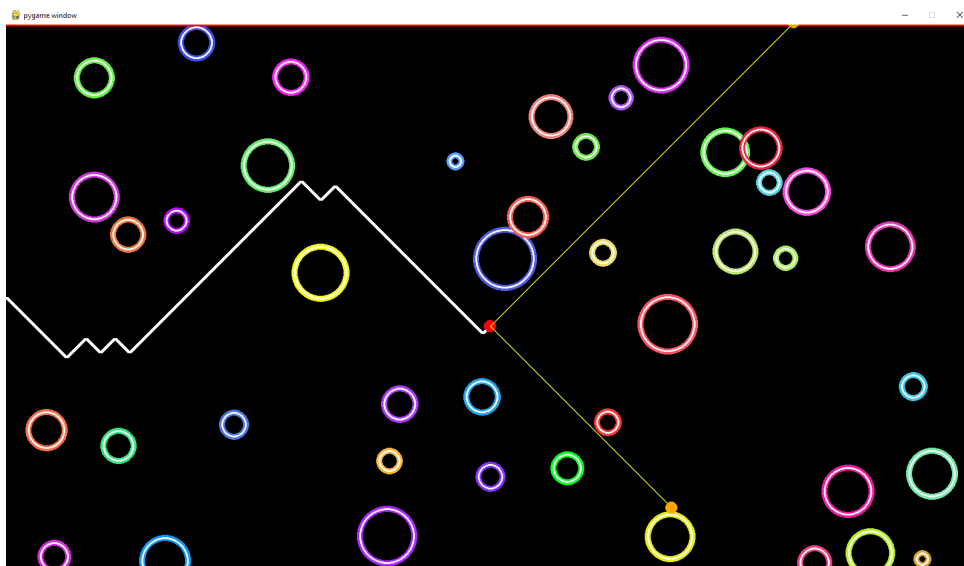


Figura 3.4: Captura del juego en funcionamiento. En este caso, las dos líneas amarillas representan la distancia hasta una colisión, teniendo en cuenta el tamaño del jugador.

Una vez que se haya jugado de la mejor manera posible durante un rato, tendremos en el mismo directorio que el juego el documento de texto con los datos etiquetados como se ve en el siguiente ejemplo:

```
1 ...
2 103.8337088717512;82.54635312476888;1
3 102.34948557129904;76.88949887527652;1
4 102.10569541383504;71.23264462578413;1
5 96.44884116434378;72.99709241423962;0
6 90.79198691485139;732.5626253092632;0
7 85.13513266535789;738.2194795587557;0
8 ...
```

Donde los dos primeros datos son las dos distancias calculadas y el tercer dato es la etiqueta (1 si el jugador dio 'click' o 0 si no hizo nada).

Entrenaremos a la red presentada al principio del Capítulo 3. Para la capa de salida, usaremos la función sigmoide, ya que lo que queremos es una variable binaria (subir o bajar). Una vez entrenada la red, en cada fotograma recogeremos los datos, pero esta vez, la salida de la red será la que tome el control del jugador.

Los resultados son satisfactorios; la red es capaz de esquivar obstáculos una cantidad decente de tiempo, como se muestra en el vídeo de YouTube:



Figura 3.5: Captura de pantalla de YouTube, donde he subido los vídeos de los resultados. URL: <https://youtu.be/AQqtye05JEA?si=2w4UFw7LzkZuwoT->

3.2. Segundo enfoque: algoritmos genéticos

En esta sección se realizará una implementación propia de un algoritmo genético para optimizar los pesos de un perceptrón multicapa. También se aplicará el algoritmo genético a un segundo juego de elaboración propia, donde evaluaremos a muchos modelos que jugarán de forma simultánea y así, de forma visual y generación tras generación, veremos cómo los jugadores van aprendiendo y consiguiendo mejores puntuaciones.

Las funciones y los métodos han sido desarrollados por mi propia cuenta, teniendo como base teórica el libro [14]. Además, algunos problemas han sido resueltos con soluciones elaboradas por mí, como se verá más adelante.

Cuando se trata de un juego de interacción en tiempo real, en el que hay más parámetros a tener en cuenta, o que una decisión que se toma pueda afectar en el futuro, es difícil que la red neuronal sea capaz de generalizar su solución a todos los casos cuando se la entrena con un conjunto de datos etiquetados, ya sea por la cantidad de datos que habría que generar para cubrir todos los casos, o porque la red neuronal se conforma con una solución local.

Es por ello que, por lo general, introducir componentes aleatorias, ya sea en los pesos, funciones de activación o incluso en la cantidad de neuronas, puede ayudar a explorar mejores soluciones, como es el caso de las redes neuronales estocásticas profundas o redes neuronales bayesianas [8].

Un método menos usado, pero muy útil cuando se trata de videojuegos, es el **aprendizaje por refuerzo**, una técnica de entrenamiento no supervisado en la que una red neuronal interactúa con un entorno y, a través de un sistema de recompensas, se le premia cuando hace algo correcto y se castiga cuando no. Generalmente, se busca maximizar una función de aptitud (**función fitness**) que, en esencia, recopila la puntuación obtenida por la red neuronal. Un sistema de recompensas mal ajustado puede dar lugar a lo que se conoce como sobreajuste (**overfitting**), por ejemplo, si las recompensas son muy altas para ciertas acciones, o penalizaciones que impidan que la red progrese a mejor ritmo.

Combinando componentes aleatorias y un sistema de recompensas surgen así los algoritmos evolutivos, en particular, los algoritmos genéticos, los cuales se basan en el modelo de evolución de las especies de Charles Darwin [15]. Combinando estos algoritmos con redes neuronales, podemos optimizar los pesos de las redes neuronales de la siguiente manera:

1. Se crea una población de N redes neuronales con parámetros aleatorios.
2. Los individuos compiten entre sí obteniendo cada uno una puntuación.
3. Se seleccionan aquellos con mejor puntuación (**élite**).
4. A partir de los individuos de la élite, se genera la siguiente generación.
5. La nueva generación vuelve a competir.

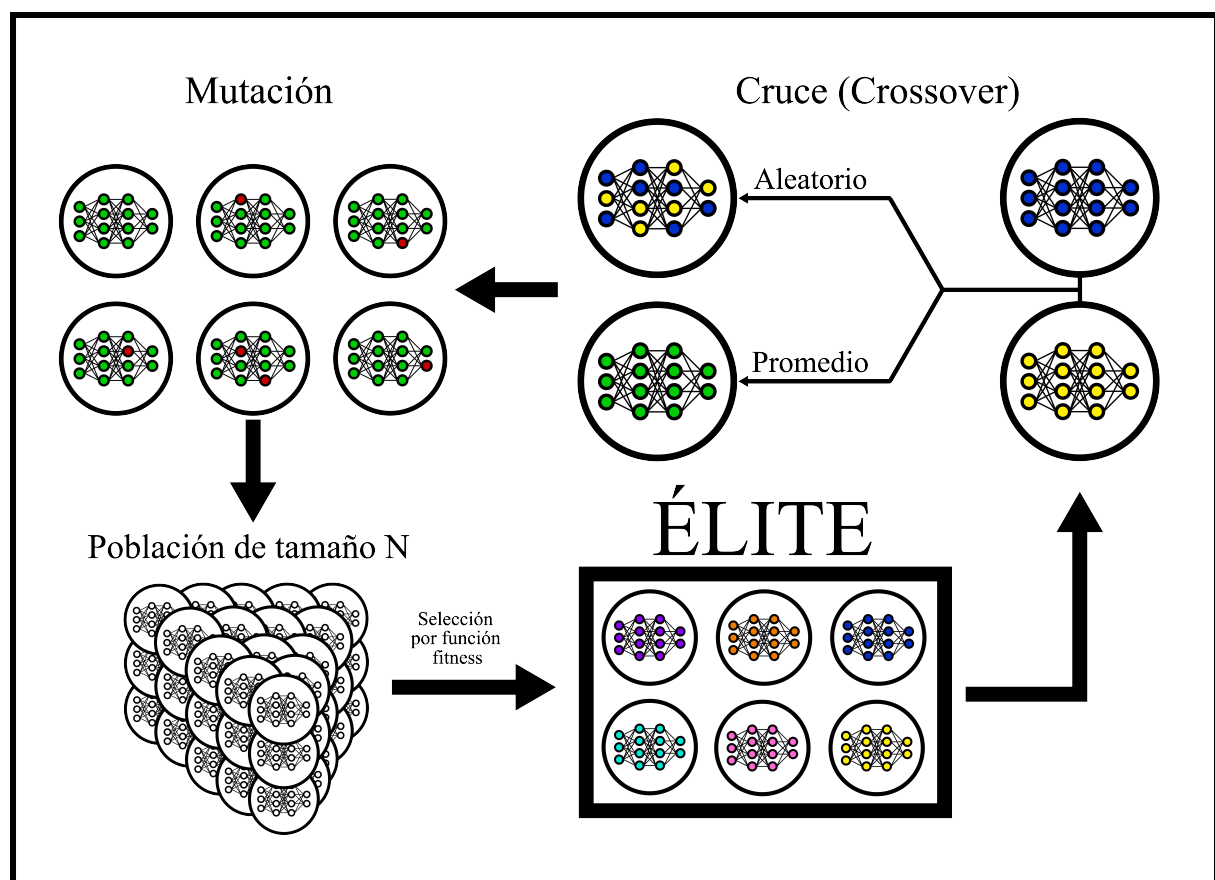


Figura 3.6: Esquema de las fases del algoritmo genético: selección, cruce y mutación.
Fuente: Elaboración propia

3.2.1. Cruce y mutación

La generación de nuevos individuos se realiza en dos fases:

- **Cruce (Crossover)**: en esta primera fase, se toman los individuos de la élite en parejas (aunque puede hacerse entre más de dos), a los que llamaremos **progenitores**, y se genera un individuo nuevo a partir de sus parámetros (en nuestro caso, los pesos y sesgo), al que llamaremos **hijo**. Ahora bien, a la hora de elegir qué parámetros tendrá el hijo, hay dos enfoques:
 - De forma **aleatoria**, siguiendo un modelo similar al de las leyes de Mendel, donde cada parámetro del hijo se copia del correspondiente parámetro de un progenitor aleatorio. Esta forma otorga mayor diversidad genética, evitando convergencias prematuras en soluciones subóptimas, aunque puede generar redes incoherentes e inestables.
 - Tomando el **promedio**, aunque no refleja bien la evolución biológica, suaviza la evolución entre generaciones y, aunque haya menos diversidad, hay más control sobre la convergencia hacia soluciones óptimas que dependiendo del azar.

Una implementación en Python utilizando las estructuras matriciales de PyTorch es la siguiente:

```

1 import random
2 def crossover(modelo1, modelo2, aleatorio=False):
3     hijo = Red()
4     pesos_padre1=modelo1.parameters()
5     pesos_padre2=modelo2.parameters
6     pesos_hijo=hijo.parameters()
7     for param1, param2, param_hijo in zip(pesos_padre1,
8 pesos_padre2,pesos_hijo):
9         if aleatorio: #aleatorio
10             if random.random() < 0.5:
11                 param_hijo.data=param1.data.clone()
12             else:
13                 param_hijo.data=param2.data.clone()
14         else: #promedio
15             param_hijo.data=(param1.data + param2.data) / 2

```

- **Mutación**: una vez tengamos los hijos de las redes de la élite, realizaremos una alteración aleatoria de algunos parámetros de cada hijo, para introducir diversidad genética y dar así paso a la evolución. Este paso es fundamental para explorar todas las posibles soluciones y evitar quedarse en óptimos locales.

Para crear una función de mutación que introduzca esos cambios aleatorios, es importante controlar cuánto queremos que varíen los parámetros; es por ello que introducimos, de manera equivalente a la tasa de aprendizaje, la **tasa de mutación**, que determinará la **probabilidad** de que determinado parámetro se modifique. Igual que la tasa de aprendizaje, una tasa de mutación elevada puede hacer que se pierda información valiosa de ciertas redes y dificulta la convergencia a la solución dada la elevada diversidad. Por el contrario, una tasa de mutación demasiado baja reduce la capacidad de exploración del espacio de soluciones y provoca una convergencia más lenta.

Una estrategia común en este caso es hacer que la tasa de mutación varíe conforme avancen las generaciones. Por lo general, una tasa de mutación elevada al principio puede ayudar a que las redes enfoquen el problema en una dirección correcta más rápidamente, y reducir esta tasa de mutación con las generaciones sirve para que las redes refinen sus soluciones.

Para el control y ajuste de la tasa de mutación existen varias estrategias [16], una bastante común es hacer que descienda de forma exponencial. Sea t la generación en la que se encuentra el algoritmo en determinado momento y $m(t)$ la tasa de mutación en la generación t :

$$m(t) = m(0) * 0,99^t$$

Aunque también, una reducción lineal o sigmoide puede servir para realizar mutaciones más suaves y controladas.

Otro factor a tener en cuenta a la hora de modificar los parámetros es la cantidad que estos parámetros se modifican. Lo más común es introducir lo que se conoce como **ruido gaussiano**, esto es, añadir un valor aleatorio generado desde una distribución normal de media 0 y desviación estándar σ , de manera que si tenemos un parámetro θ , la actualización del parámetro a θ' es:

$$\theta' = \theta + N(0, \sigma)$$

Aunque también es posible utilizar otras distribuciones de probabilidad (como la uniforme) según el problema al que se esté haciendo frente. Finalmente, la implementación en Python de la función *mutar*, con una tasa de mutación inicial $m(0) = 0,1$ y una desviación típica $\sigma = 0,1$ nos queda:

```
1 def mutar(red, generacion=0):
2     m_0 = 0.1
3     sigma=0.1
4     tasa_mutacion = m_0 * 0.99 ** generacion
5     with torch.no_grad():
6         for param in red.parameters():
7             valores=param.data.view(-1)
8             for i in range(param.numel()):
9                 if random.random() < tasa_mutacion:
10                    valores[i] += random.gauss(0,sigma)
```

PyTorch trabaja con tensores, estructuras de datos optimizadas para realizar todas las combinaciones de operaciones de una capa a otra en una red neuronal. Al entrenar un modelo mediante propagación hacia atrás, la librería guarda un historial de operaciones para calcular los gradientes de manera eficiente. Sin embargo, en un algoritmo genético no necesitamos calcular gradientes, por lo que en la línea 5 usamos `torch.no_grad()`, indicando que no se almacenen estas operaciones, reduciendo así el uso de memoria.

Además, en la línea 7, usamos `param.data.view(-1)`, que convierte el tensor en un vector unidimensional de punteros al propio tensor. Esto nos permite recorrer y modificar cada valor de los parámetros del modelo de manera independiente, según la tasa de mutación.

Como los individuos que no forman parte de la élite son eliminados, es necesario que los procesos de cruce y mutación generen suficientes nuevos ejemplares para recuperar el tamaño original de la población. Es decir, si cada generación tiene un tamaño N y se seleccionan los N_{ELITE} mejores individuos para formar la élite, se necesitará crear $N - N_{ELITE}$ nuevos ejemplares. De esta forma, la élite pasará a la siguiente generación sin ser eliminada, garantizando que los mejores individuos se conservan si no son superados en generaciones posteriores.

También es necesario preservar la diversidad en la creación de una nueva generación, para una mejor exploración del espacio de soluciones. Para ello, es importante gestionar bien la elección de combinaciones de progenitores de la élite para realizar los cruces, y no dejar a ningún miembro de la élite con menos descendientes que otros.

Lo ideal sería reproducir todas las posibles parejas de progenitores, es decir, todas las combinaciones de elementos tomados de 2 en 2:

$$\binom{N_{ELITE}}{2} = \frac{N_{ELITE} \cdot (N_{ELITE} - 1)}{2}$$

Si queremos realizar todas las posibles combinaciones preservando la élite para la siguiente generación, el valor que necesitaríamos para N en función de N_{ELITE} es:

$$N = \frac{N_{ELITE} \cdot (N_{ELITE} - 1)}{2} + N_{ELITE} = \frac{N_{ELITE} \cdot (N_{ELITE} + 1)}{2}$$

Este valor nos da una proporción bastante inestable al variar N_{ELITE} . Por ejemplo, tomando $N_{ELITE} = 15$ necesitaríamos generaciones de $N = 120$ individuos, pero si queremos una élite de $N_{ELITE} = 80$, necesitaríamos una población de $N = 3240$.

Como consecuencia, para un mayor control de la proporción de la población que entra en la élite, y reducir el coste computacional que supondría gestionar todas las combinaciones posibles, es más cómodo elegir los valores N y N_{ELITE} . Esto tiene el inconveniente de no poder seleccionar todas las combinaciones, lo que nos obliga a quedarnos con menos o, en casos en los que la élite es más pequeña, a repetir algunas para ajustarse al valor de N .

Al no poder tener todas las posibles combinaciones o, en algunos casos, faltarnos combinaciones, debemos elegir combinaciones de forma que, cuando se acabe la cantidad de vacantes a nuevos individuos, todos los miembros de la élite hayan aportado una cantidad similar de hijos.

Para entender el problema, observemos el siguiente ejemplo. Supongamos que la élite está formada por las letras de la lista de tamaño $N_{ELITE} = 10$ y queremos emparejarlos para formar N descendientes:

```
1 lista=['a','b','c','d','e','f','g','h','i','j']
```

La manera convencional de hacerlo es con un sencillo doble bucle:

```
1 combinaciones=[]
2 for i in range(len(lista)):
3     for j in range(i+1,len(lista)):
4         combinaciones.append([lista[i],lista[j]])
```

Si tomásemos solo las $N = 20$ primeras combinaciones:

```
1 [['a', 'b'], ['a', 'c'], ['a', 'd'], ['a', 'e'], ['a', 'f'],
2 ['a', 'g'], ['a', 'h'], ['a', 'i'], ['a', 'j'], ['b', 'c'],
3 ['b', 'd'], ['b', 'e'], ['b', 'f'], ['b', 'g'], ['b', 'h'],
4 ['b', 'i'], ['b', 'j'], ['c', 'd'], ['c', 'e'], ['c', 'f']]
```

La cantidad de descendientes de cada elemento es:

Letra	a	b	c	d	e	f	g	h	i	j
Descendientes	9	9	5	3	3	3	2	2	2	2

Cuadro 3.1: Cantidad de descendientes de cada letra

Los elementos ‘a’ y ‘b’ tienen bastantes más descendientes que el resto por el orden en el que se generan las combinaciones. Para balancear la cantidad de descendientes de cada miembro de la élite, utilizaremos el siguiente orden para recoger N combinaciones de los elementos de la lista tomados de dos en dos:

```
1 lista=['a','b','c','d','e','f','g','h','i','j']
2 combinaciones=[]
3 indice=0
4 salto=1
5 N=20
6 while len(combinaciones)<N:
7     siguiente=(indice+salto)%len(lista)
8     combinaciones.append([lista[indice],lista[siguiente]])
9     indice+=1
10    indice%=len(lista)
11    if indice==0:
12        salto+=1
```

Además, como hemos mencionado, si la cantidad total de combinaciones junto a la élite, anteriormente es:

$$\hat{N} = \frac{N_{ELITE} \cdot (N_{ELITE} + 1)}{2}$$

Este algoritmo funciona tanto si $N \leq \hat{N}$ como si $N \geq \hat{N}$, en este segundo caso, ciclando sobre las mismas combinaciones hasta llegar a N ya que se reinician los valores de los índices automáticamente cuando se completan todas las combinaciones.

De esta forma, tomando las 20 primeras combinaciones que nos da el algoritmo:

```
1 [['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'e'], ['e', 'f'],
2 ['f', 'g'], ['g', 'h'], ['h', 'i'], ['i', 'j'], ['j', 'a'],
3 ['a', 'c'], ['b', 'd'], ['c', 'e'], ['d', 'f'], ['e', 'g'],
4 ['f', 'h'], ['g', 'i'], ['h', 'j'], ['i', 'a'], ['j', 'b']]
```

Como podemos observar, este algoritmo nos ayudará a preservar la diversidad al no darle prioridad a ningún elemento de la élite a causa del diseño del algoritmo, haciendo que todos los individuos tengan una cantidad similar de descendientes.

Letra	a	b	c	d	e	f	g	h	i	j
Descendientes	4	4	4	4	4	4	4	4	4	4

Cuadro 3.2: Frecuencia de cada letra

Al final de cada generación, crearemos la siguiente utilizando todas las funciones definidas en esta sección:

```

1 hijos = []
2 indice=0
3 salto=1
4 while len(hijos) + elite_n < N:
5     siguiente=(indice+salto)%elite_n
6     padre1, padre2 = elite[indice], elite[siguiente]
7     hijo=crossover(padre1, padre2, generacion=gen, aleatorio=False)
8     #La funcion de mutacion se aplica dentro de crossover
9     hijos.append(hijo)
10    indice+=1
11    indice%=elite_n
12    if indice==0:
13        salto+=1
14 models=elite+hijos

```

3.2.2. Implementación

A continuación, se expondrá el caso práctico que estudiaremos, los problemas afrontados y los resultados. En esta ocasión, intentaremos que la red aprenda a superar otro juego de elaboración propia.

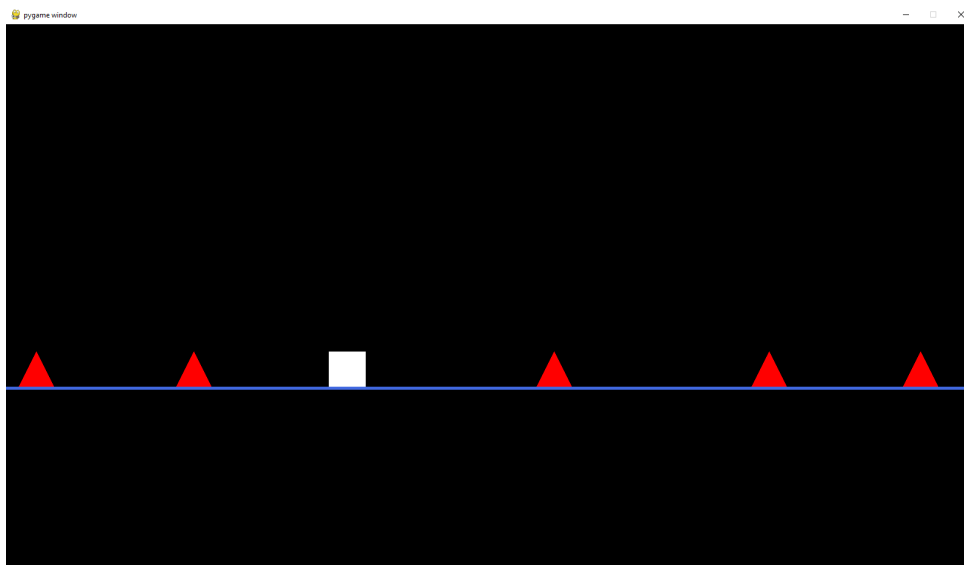


Figura 3.7: Captura de pantalla del segundo juego, el cuadrado se encuentra en el suelo, deslizándose por el suelo y deberá saltar los obstáculos triangulares según se acercan.

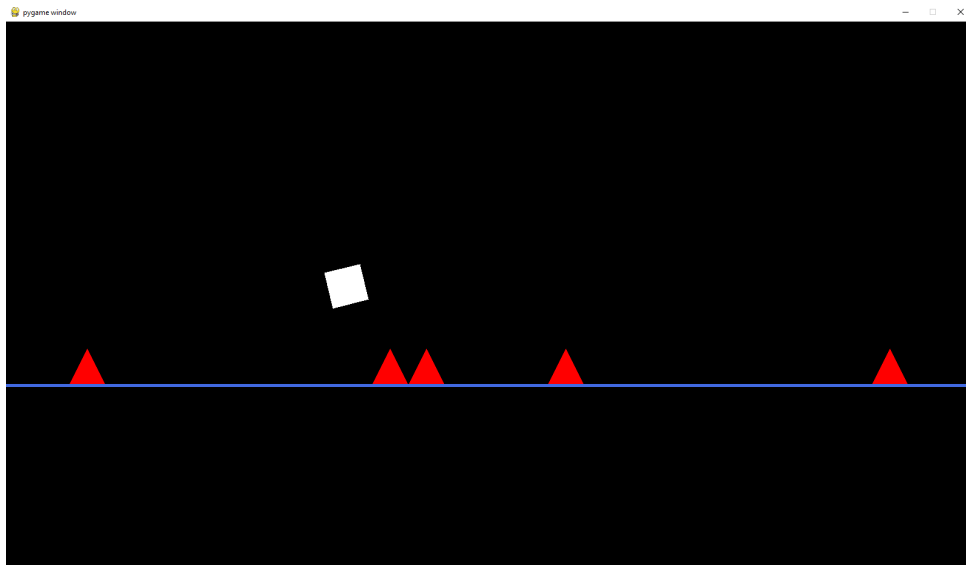


Figura 3.8: Captura de pantalla del segundo juego, en este caso, el jugador ha saltado para esquivar los obstáculos haciendo que estos pasen por debajo.

El jugador es un cuadrado blanco que se desliza por el suelo avanzando automáticamente hacia la derecha, y debe saltar los obstáculos con forma de triángulo pulsando ‘click’. En ocasiones, los obstáculos pueden aparecer unidos, como se puede ver en la imagen [3.8](#) y separados, como mínimo, el ancho del cuadrado del jugador (para que se pueda saltar entre dos obstáculos si se precisa).

Nuestro objetivo es entrenar una serie de redes neuronales, en una primera generación, con pesos aleatorios y, mediante el algoritmo genético, ir seleccionando a aquellos que consigan mejor puntuación para que tanto ellos como sus hijos pasen a la siguiente generación.

A la hora de elegir los datos con los que operan las redes neuronales, podríamos optar por la distancia al siguiente obstáculo. El problema es que la generación de obstáculos hace que el juego no sea tan sencillo como, simplemente, saltar cuando se aproxima el siguiente obstáculo, sino que se deben considerar las distancias a todos los obstáculos para ciertos casos:

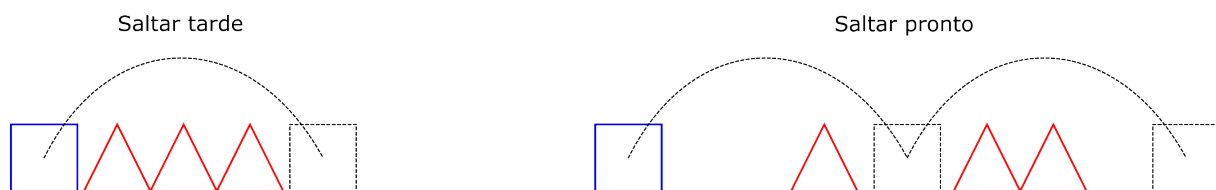


Figura 3.9: Esquema de dos casos diferentes en el juego. Lo máximo que puede saltar el jugador son 3 obstáculos seguidos apurando lo máximo posible el salto. Sin embargo, hay casos en los que eso no conviene y debe saltar antes para no colisionar contra otro obstáculo.

Estos son algunos de los muchos casos en los que es necesario tener en cuenta la información de todos los obstáculos y, para que sean capaces de reconocer si están en el suelo o en el aire, también es necesario aportar la altura del jugador como dato. Por este motivo, a diferencia del juego anterior, donde las dos variables de entrada tenían escalas o rangos de valores similares, aquí no es el caso, dado que la altura del jugador es una variable muy di-

ferente de las distancias a los obstáculos. En estas circunstancias, es necesario normalizar los datos de entrada al intervalo $[0, 1]$.

Una cuestión que surge de aquí es, de qué manera organizar los datos que reciben, y si la entrada cambiará de tamaño según la cantidad de obstáculos. Para hacer frente a este problema, es necesario entender cómo funciona el sistema de obstáculos. En primer lugar, por eficiencia, realmente en el juego solo hay 6 obstáculos que se ciclan de manera dinámica; cuando un obstáculo desaparece por la izquierda, se desactiva y espera en una cola a ser activado de nuevo, instante en el que su posición es actualizada a la posición inicial (a la derecha de la pantalla).

Si en la entrada a la red ponemos las distancias a cada obstáculo, habrá distancias negativas (los obstáculos que quedan a la izquierda) y cambios drásticos al actualizar los obstáculos (al hacerlos reaparecer). Por ello, descartamos los valores negativos, pero esto haría que la entrada cambiase de tamaño según el momento en el que se encuentre, así que, cada vez que un obstáculo sea superado por el jugador, el dato de la distancia se actualizará a 1; como se normalizan los datos, estar a distancia 1 equivale a estar en el punto de aparición (a la derecha de la pantalla), aunque realmente no se encuentre ahí.

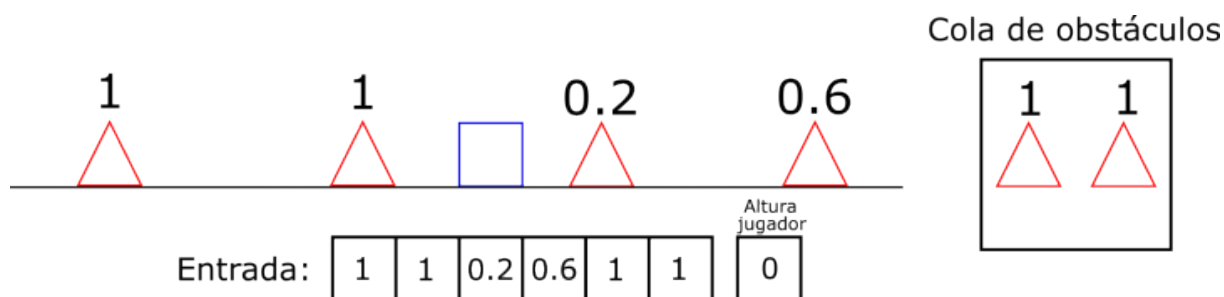


Figura 3.10: Esquema de la entrada que recibe la red neuronal, una casilla asignada a cada obstáculo y la altura del jugador

Ahora bien, teniendo en cuenta la arquitectura del perceptrón multicapa, la posición en la entrada es la única manera de distinguir entre una variable y otra. En vez de ordenar en la entrada los obstáculos por su índice en la memoria del código (según el orden en el que fueron creados), es mejor ordenarlos por proximidad (de menor a mayor). De esta forma, en lugar de obtener la distancia a cada uno de los 6 obstáculos, le proporcionamos las distancias a los siguientes 6 obstáculos.

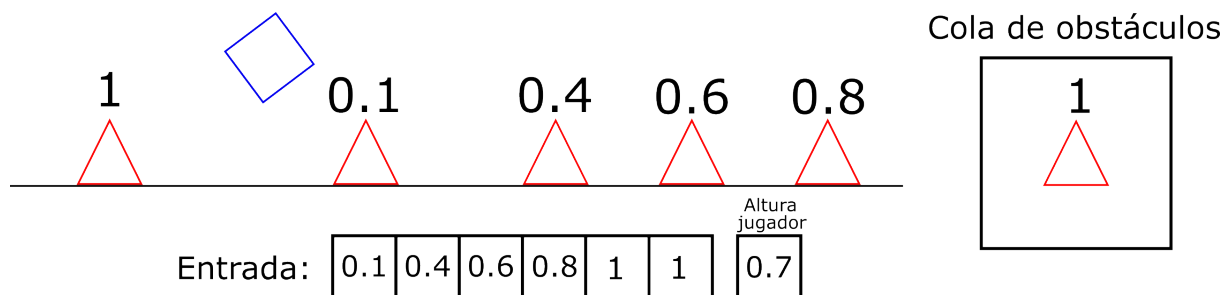


Figura 3.11: Esquema de la entrada que recibe la red neuronal, una casilla para las distancias a los obstáculos siguientes y la altura del jugador

Una vez decidida la entrada, podemos proceder con el algoritmo genético. Para el cruce, tomaremos el promedio de los pesos, y utilizaremos la función de mutación definida anteriormente con $m_0 = 0,1$, y las mutaciones se realizarán añadiendo un valor aleatorio de distribución $N(0, \sigma)$ con $\sigma = 0,1$.

Una vez readaptado el código del juego para admitir múltiples jugadores al mismo tiempo, ejecutamos una primera generación de $N = 500$ individuos y una élite de tamaño 20. Cada individuo contará con un perceptrón multicapa de una capa de entrada de tamaño 7 (una por variable de entrada), tres capas ocultas de tamaños 10, 10 y 7 con función de activación ReLU, y una capa de salida de una sola neurona con la función sigmoide, que decidirá si el jugador salta o no en cada fotograma.

La función fitness es bastante sencilla; un punto por cada fotograma vivo, y $-0,1$ puntos cada vez que un individuo salte estando en el aire, ya que carece de sentido, y así los individuos que salten constantemente (como ocurre en las primeras generaciones), y esquiven algunos obstáculos antes de que su suerte finalice, no obtengan una puntuación muy superior a aquellos que identificaron y saltaron un obstáculo (pero, por imprecisiones, colisionen contra otro).

No es necesario quitar puntuación por colisionar, dado que tarde o temprano, todos colisionarán. En este vídeo se puede apreciar la primera generación, donde individuos que saltan siempre llegan más lejos: https://youtu.be/T1e0Q19yv6U?si=7gv_3Gcr27tEBHHf

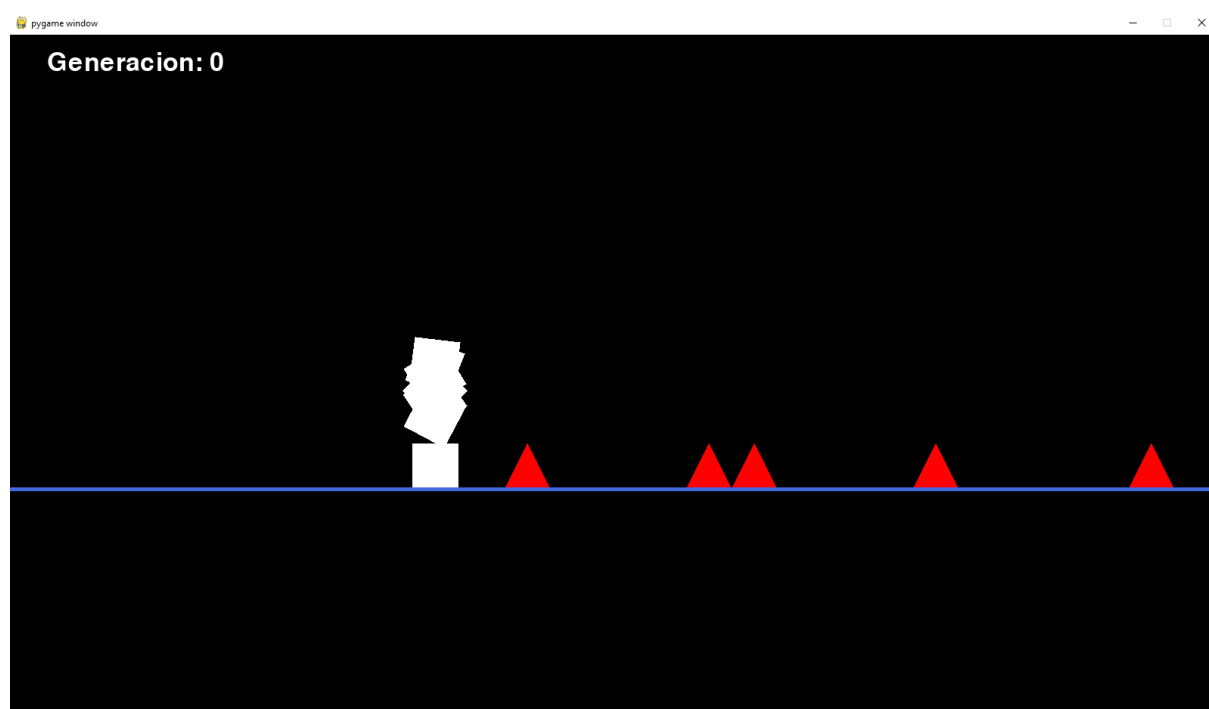


Figura 3.12: Captura de la generación 0 del algoritmo, cada individuo hace una acción muy diferente.

Al ejecutar el código `generaciones.py` del repositorio de GitHub comienza la primera generación seguida de todas las demás. En este enlace a YouTube hay un vídeo con una muestra de más de 40 generaciones donde se puede apreciar la evolución de los jugadores: <https://youtu.be/gXfjCB0shns?si=I21ECcQVks-GCq5S>

Como era de esperar, en la primera generación, la diversidad hace que cada individuo haga una cosa diferente en cada fotograma, la mayoría, sin sentido. Algunos optan por saltar siempre que pueden, otros no saltan y otros saltan en momentos diferentes, siendo aquellos que saltan constantemente los que tienen mayor tasa de éxito.

No obstante, dejando el algoritmo funcionar generación tras generación, podemos observar cómo la aleatoriedad y la selección artificial se abren camino, dando lugar a individuos que pronto empiezan a identificar y saltar obstáculos.

En unas primeras generaciones (0-25) observamos que reconocen los obstáculos y los saltan, aunque cuando salen dos obstáculos seguidos, muchos saltan demasiado pronto y otros no llegan a saltar. También hay ocasiones en que, habiendo un solo obstáculo, colisionan con él por no tener, posiblemente, una percepción total del tamaño de los obstáculos ni del jugador.

Alrededor de la generación 25 se puede apreciar cómo controlan ya el tamaño de las colisiones, es raro encontrar individuos que mueran colisionando contra un obstáculo por no saltar a tiempo, casi todos saltan. Algunos saltan muy temprano y caen rozando el obstáculo, mientras que otros saltan lo más tarde posible. La idea a partir de aquí es que aprendan a ajustar el momento en el que deben saltar según lo necesiten, dado que saltar siempre pronto o siempre tarde no es una estrategia que sirva a largo plazo.

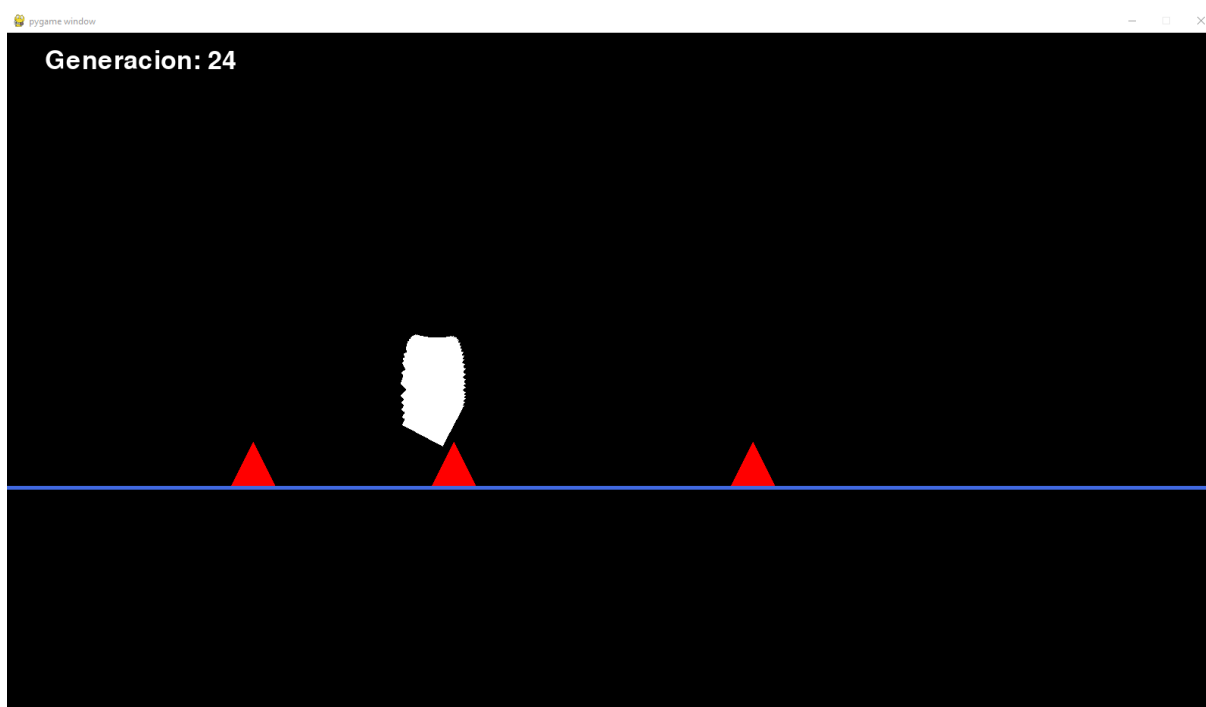


Figura 3.13: Captura de la generación 24 donde todos los individuos saltan el mismo obstáculo y la diferencia entre unos y otros es menor, explorando así mejor las posibles soluciones.

Finalmente, tras varias generaciones (más de 120) conseguimos unos modelos capaces no solo de saltar los obstáculos que se aproximan, sino también de tener que realizar su salto temprano o apurarlo más según sea conveniente, teniendo en cuenta las distancias con los demás obstáculos. En algunos casos, incluso, podemos apreciar un desarrollo de estrategias para resolver casos más complejos como este caso en concreto, resuelto con éxito varias veces durante las pruebas:

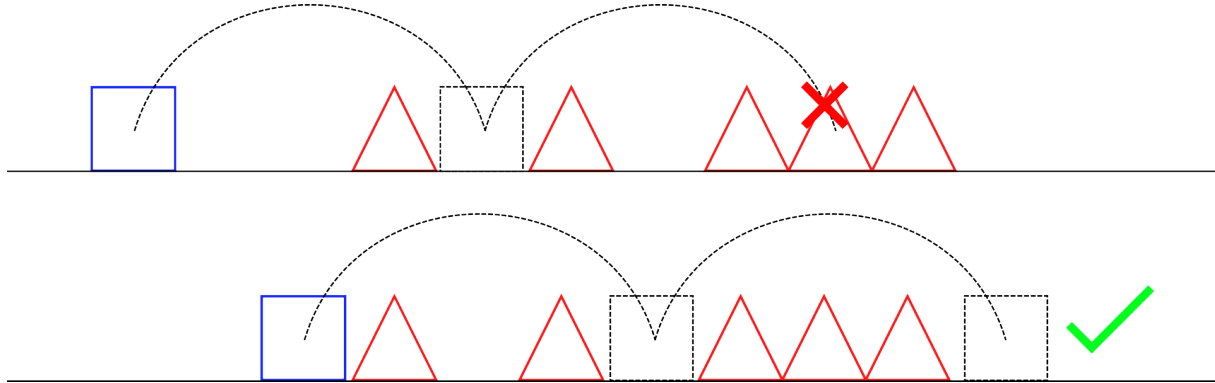


Figura 3.14: Esquema de un caso bastante difícil superado con éxito por algunos ejemplares, el individuo debe mostrarse capaz de elegir la estrategia que le permite sobrevivir.

Capítulo 4

Conclusiones y Trabajo Futuro

En este último capítulo, se pretende dar una visión global de las conclusiones a las que se ha llegado durante el desarrollo de este trabajo, así como abrir nuevos horizontes para nuevos proyectos en el futuro, relativos a los contenidos de esta memoria.

4.1. Conclusiones

En esta memoria no solo se ha realizado un recorrido de los fundamentos de las redes neuronales y sus implementaciones en Python, sino que se ha profundizado en los conceptos de los algoritmos genéticos para una mejor comprensión de los mismos y de la teoría de la evolución de las especies.

Se ha podido observar que no es necesario estar en disposición de unos datos de entrenamiento que, en el caso de los juegos de interacción en tiempo real, no suelen estar fácilmente disponibles. Por lo general, cuando se quiere entrenar a un modelo para que aprenda a jugar a uno de estos juegos, la cantidad de datos que se necesitan para ciertos casos puede ser extremadamente grande y aún así, no conseguir unos modelos que hayan encontrado la solución óptima. También es posible que, en juegos complejos, se desconozcan las etiquetas correspondientes a cada dato.

Por eso, los algoritmos genéticos, al igual que la evolución de las especies, se abren camino explorando todas las posibles soluciones. Además cabe notar la diferencia presente en este trabajo a la hora de tratar los datos. Por un lado, para el entrenamiento supervisado (con el optimizador Adam), se ha necesitado llevar a cabo un detallado estudio del problema para asegurarnos de que los datos que se le proporcionaban a la red eran lo suficientemente buenos como para que encontrara una solución rápidamente. Por otro lado, para el caso del algoritmo genético, ha sido suficiente utilizar como datos, las distancias a los obstáculos sin la necesidad de hacer ningún cálculo más complejo que pudiera aportar más información.

Finalmente, en el caso del segundo juego, podemos observar en las últimas generaciones cómo, al igual que las abejas al nacer ya estaban ‘programadas’ para realizar algunas tareas, los modelos de las últimas generaciones conocían a la perfección el objetivo del juego.

4.2. Trabajo futuro

Hay algunos aspectos del trabajo que pueden ser modificados para obtener unos resultados diferentes, que sería interesante investigar.

En primer lugar, tanto en el primero como en el segundo juego, existen unos parámetros que dependen de la aleatoriedad (generación de obstáculos). Esto hace que algunos resultados sean susceptibles de mejora. En particular, he realizado un análisis sobre las puntuaciones en el segundo juego de 250 generaciones. En este análisis se ve cómo progresa la media del tiempo que están vivos los ejemplares de una generación, medido en segundos:

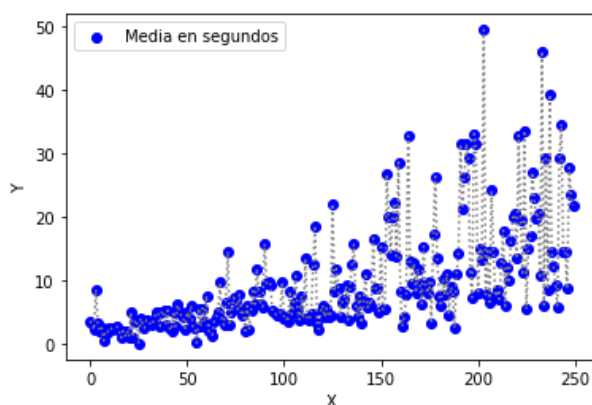


Figura 4.1: Gráfico de la media del tiempo que están activos los individuos en segundos (vertical) en cada generación (horizontal)

Si calculamos la recta de regresión:

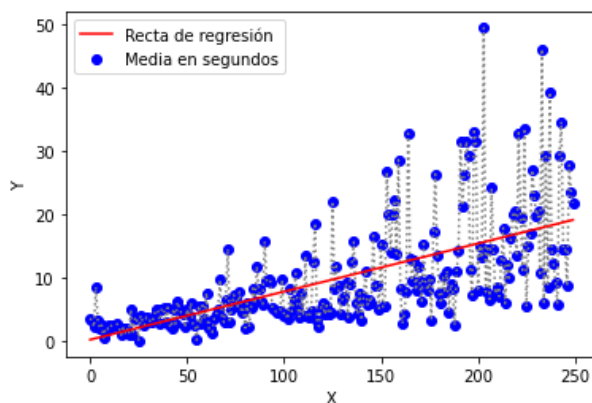


Figura 4.2: Mismo gráfico que el anterior, pero con la recta de regresión

Efectivamente, la media progresa, no obstante, ese factor aleatorio de la generación de obstáculos hace que la dispersión sea muy elevada. Esto se debe la posibilidad de generar casos difíciles a los que las generaciones no han hecho frente con anterioridad y, aunque estas se vayan superando poco a poco, dificulta mucho el progreso.

De cara a posibles soluciones de este problema, un posible trabajo futuro sería utilizar algoritmos genéticos pero para **redes neuronales convolucionales**. Este tipo de red neuronal es perfecta para el tratamiento de estructuras de datos en forma de cuadrícula, en particular, imágenes. Esto sería útil porque no solo evitaría la tarea de tener que reprogramar los juegos al tener acceso a las imágenes de los videjuegos, sino que también puede facilitar la resolución de los casos difíciles, por ejemplo, del segundo juego.

Otro posible cambio que se podría llevar a cabo es el uso de **neuroevolución de topologías aumentadas**, que es en lo que se basa la librería Neat. Esto permite modificar la estructura de la red neuronal como parte del proceso del algoritmo genético. Es similar al funcionamiento del método de caída de Montecarlo (*dropout*), solo que en lugar de ‘apagar’ neuronas de forma aleatoria, modifica la estructura de la red neuronal, entre otras cosas.

Estos cambios podrían ser fruto de investigación, quizá, partiendo de la ya construida librería Neat para facilitar el proceso. De esta forma, se obtiene una manera interesante y visual de entrenar modelos para que jueguen a videojuegos.

Bibliografía

- [1] César Ades. *El origen del instinto animal*. 2009. URL: <https://noticiasdelaciencia.com/art/49803/el-origen-del-instinto-animal>
- [2] Code Reclaimers. *NEAT-Python: NeuroEvolution of Augmenting Topologies implementation in Python*. URL: <https://neat-python.readthedocs.io/en/latest/>
- [3] E. Rollings A. Adams. *Fundamentals of Game Design*. Prentice Hall. 2003.
- [4] Pygame Community. *Pygame: A set of Python modules designed for writing video games*. URL: <https://www.pygame.org/docs/>
- [5] Fran Ramirez. “Historia de la IA: Frank Rosenblatt y el MarkI Perceptrón”. En: (2018). URL: <https://telefonicatech.com/blog/historia-de-la-ia-frank-rozenblatt-y-e>
- [6] Jakob Stefan. “Das Perceptron: Urgestein der modernen KI”. En: (2024). URL: <https://www.micromata.de/blog/perceptron-ki/>
- [7] Sidhart. “Multi-Layer Perceptron Explained: A Beginner’s Guide”. En: (2023). URL: <https://www.pycodemates.com/2023/01/multi-layer-perceptron-a-complete-overview.html>
- [8] D. Aragón Santín. “TFG- Introducción a las redes neuronales bayesianas.” En: (2024).
- [9] GeeksforGeeks. “Activation functions in Neural Networks”. En: (2024). URL: <https://www.geeksforgeeks.org/activation-functions-neural-networks/>
- [10] InteractiveChaos. “Aprendizaje batch y online”. En: (). URL: <https://interactivechaos.com/es/manual/tutorial-de-machine-learning/aprendizaje-batch-y-online>
- [11] Richmond Alake. “Explicación de las funciones de pérdida en el machine learning”. En: (). URL: <https://www.datacamp.com/es/tutorial/loss-function-in-machine-learning>
- [12] Yoshua Bengio y Aaron Courville. “Deep Learning, Chapter 6”. En: (). URL: <https://www.deeplearningbook.org/contents/mlp.html>
- [13] PyTorch. *An Imperative Style, High-Performance Deep Learning Library*. 2019. URL: <https://pytorch.org/docs/stable/index.html>
- [14] Oliver Kramer. *Genetic algorithm essentials*. Springer, 2017.
- [15] Construyendo a Chispas. *Bases biológicas de los algoritmos genéticos y evolutivos*. 2019. URL: <https://construyendoachispas.blog/2019/12/08/bases-biologicas-de-los-algoritmos-geneticos-y-evolutivos/>

- [16] Martin S. Krejca y Carsten Witt. “A Flexible Evolutionary Algorithm With Dynamic Mutation Rate Archive”. En: (2024). URL: <https://arxiv.org/abs/2404.04015>.

Anexos

Info Anexo 1

Info anexo 2

Info anexo 3